

```
In [ ]: !pip install -q pyserial dynamixel-sdk numpy matplotlib
```

```
In [ ]: import time
import math
import re
from collections import deque

import serial
import numpy as np
import matplotlib.pyplot as plt

from dynamixel_sdk import *

print("Imports OK")
```

```
In [ ]: ARDUINO_PORT = "COM3"
ARDUINO_BAUD = 9600
ARDUINO_TIMEOUT_S = 0.05

SENSOR_PATTERN = re.compile(r"raw:\s*(-?\d+)")

SENSITIVITY_N_PER_COUNT = 4.8e-5
SENSOR_SIGN = 1.0

FILTER_WINDOW = 2
TARE_SAMPLES = 20

DXL_ID = 1
DXL_PORT = "COM4"
DXL_BAUD = 57600
PROTOCOL_VERSION = 2.0

ADDR_OPERATING_MODE = 11
ADDR_TORQUE_ENABLE = 64
ADDR_GOAL_PWM = 100
ADDR_PRESENT_VELOCITY = 128
ADDR_PRESENT_POSITION = 132
ADDR_PRESENT_INPUT_VOLTAGE = 144
ADDR_PWM_LIMIT = 36

PWM_CONTROL_MODE = 16
TORQUE_ENABLE = 1
TORQUE_DISABLE = 0

PWM_FULL_SCALE = 885

MAX_PWM = 300

MIN_EFFECTIVE_PWM_FORWARD = 25
MIN_EFFECTIVE_PWM_BACKWARD = 65

MIN_POS_TICKS = 2300
MAX_POS_TICKS = 3500
```

```

USE_POSITION_COMP = True
CENTER_POS_TICKS = 3000
KG_POS = 0.08

TARGET_TENSION_N = 3.5
TENSION_DEADBAND_N = 0.10

CONTROL_FREQ_HZ = 20.0
MAX_DURATION_S = 20.0

LOW_TENSION_POS_GAIN_TICKS_PER_N = 400.0
HIGH_TENSION_POS_GAIN_TICKS_PER_N = 160.0

MAX_TARGET_OFFSET_TICKS = 240
TARGET_ALPHA = 0.30
MAX_TARGET_STEP_TICKS = 30

RESET_TARGET_ON_REGION_REVERSAL = True

KP_POS_TO_PWM = 0.70
KD_VEL_TO_PWM = 30.0

POSITION_ERROR_DEADBAND_TICKS = 12
MIN_PWM_ENABLE_ERROR_TICKS = 35

MAX_PWM_STEP = 30

HIGH_TENSION_RELEASE_PWM_GAIN = 16.0
HIGH_TENSION_RELEASE_PWM_MAX = 45
HIGH_TENSION_RELEASE_MIN_PWM = 15

BITE_TRIGGER_TENSION_N = 0.8
BITE_CONFIRM_TIME_S = 0.10

LOW_TENSION_RECOVERY_N = 1.0
TENSION_RECOVERED_N = 2.0
LOW_TENSION_TIMEOUT_S = 1.8
SOFT_BACK_LIMIT_TICKS = 2600

print("Config loaded")

```

```

In [ ]: class FixedFrequencyLoopManager:
        def __init__(self, frequency_Hz):
            self.period_s = 1.0 / float(frequency_Hz)
            self._next_tick = None

        def sleep(self):
            now = time.time()

            if self._next_tick is None:
                self._next_tick = now + self.period_s
                return

            sleep_time = self._next_tick - now

            if sleep_time > 0:

```

```

        time.sleep(sleep_time)

        self._next_tick += self.period_s

def parse_sensor_line(line):
    match = SENSOR_PATTERN.search(line)

    if not match:
        return None

    return int(match.group(1))

def signed_velocity_to_rad_s(vel_raw):
    if vel_raw >= (1 << 31):
        vel_raw -= (1 << 32)

    return vel_raw * 0.229 * 2.0 * math.pi / 60.0

def clip_pwm(pwm):
    return int(max(-MAX_PWM, min(MAX_PWM, pwm)))

def apply_min_effective_pwm(pwm):
    pwm = int(pwm)

    if pwm > 0:
        return max(pwm, MIN_EFFECTIVE_PWM_FORWARD)

    if pwm < 0:
        return min(pwm, -MIN_EFFECTIVE_PWM_BACKWARD)

    return 0

def position_comp_pwm(pos):
    if not USE_POSITION_COMP:
        return 0

    comp = KG_POS * (CENTER_POS_TICKS - pos)
    return int(round(comp))

def signed_pwm_to_unsigned(pwm):
    pwm = int(pwm)

    if pwm < 0:
        return pwm + (1 << 16)

    return pwm

print("Helpers ready")

```

```

In [ ]: class LoadCellSerialReader:
    def __init__(
        self,
        port,
        baud,
        timeout_s=0.05,
        filter_window=2,
        tare_samples=20,
        sensitivity_N_per_count=4.8e-5
    ):
        self.port = port
        self.baud = baud
        self.timeout_s = timeout_s
        self.filter_window = filter_window
        self.tare_samples = tare_samples
        self.sensitivity = sensitivity_N_per_count

        self.ser = None
        self.buffer = deque(maxlen=filter_window)
        self.zero_offset = None

    def open(self):
        self.ser = serial.Serial(
            self.port,
            self.baud,
            timeout=self.timeout_s
        )

        time.sleep(2.0)
        self.ser.reset_input_buffer()

        print(f"Arduino connected on {self.port}")
        print("Taring load cell. Do not pull the line...")

        self.zero_offset = self.tare()

        print(f"Tare complete. zero_offset = {self.zero_offset:.1f}")

    def read_raw_once(self):
        while True:
            line = self.ser.readline().decode("utf-8", errors="ignore").stri

            if not line:
                continue

            raw = parse_sensor_line(line)

            if raw is not None:
                return raw, line

    def tare(self):
        samples = []

        while len(samples) < self.tare_samples:
            raw, _ = self.read_raw_once()

```

```

        samples.append(raw)

    return float(np.mean(samples))

def raw_to_tension(self, filtered_raw):
    return SENSOR_SIGN * self.sensitivity * (filtered_raw - self.zero_of

def read(self):
    line = self.ser.readline().decode("utf-8", errors="ignore").strip()

    if not line:
        return None, None, None, line

    raw = parse_sensor_line(line)

    if raw is None:
        return None, None, None, line

    self.buffer.append(raw)

    filtered_raw = float(np.mean(self.buffer))
    tension_N = self.raw_to_tension(filtered_raw)

    return raw, filtered_raw, tension_N, line

def close(self):
    if self.ser is not None:
        self.ser.close()
        print("Arduino serial closed")

```

```

In [ ]: class DynamixelPWMInterface:
    def __init__(self, dxl_id, port, baud):
        self.dxl_id = dxl_id
        self.port = port
        self.baud = baud

        self.portHandler = PortHandler(self.port)
        self.packetHandler = PacketHandler(PROTOCOL_VERSION)

    def open(self):
        if not self.portHandler.openPort():
            raise RuntimeError(f"Failed to open Dynamixel port {self.port}")

        if not self.portHandler.setBaudRate(self.baud):
            raise RuntimeError(f"Failed to set Dynamixel baudrate {self.baud}")

        print(f"Dynamixel port opened: {self.port}")

        _, comm_result, _ = self.packetHandler.ping(
            self.portHandler,
            self.dxl_id
        )

        if comm_result != COMM_SUCCESS:
            raise RuntimeError(f"Failed to ping Dynamixel ID {self.dxl_id}")

```

```

        print(f"Dynamixel ID {self.dxl_id} connected")

    def set_pwm_limit(self, limit=885):
        self.packetHandler.write1ByteTxRx(
            self.portHandler,
            self.dxl_id,
            ADDR_TORQUE_ENABLE,
            TORQUE_DISABLE
        )

        comm, err = self.packetHandler.write2ByteTxRx(
            self.portHandler,
            self.dxl_id,
            ADDR_PWM_LIMIT,
            int(limit)
        )

        pwm_limit, _, _ = self.packetHandler.read2ByteTxRx(
            self.portHandler,
            self.dxl_id,
            ADDR_PWM_LIMIT
        )

        print("set PWM limit:", pwm_limit, "comm:", comm, "err:", err)

    def enable_pwm_mode(self):
        self.packetHandler.write1ByteTxRx(
            self.portHandler,
            self.dxl_id,
            ADDR_TORQUE_ENABLE,
            TORQUE_DISABLE
        )

        self.packetHandler.write1ByteTxRx(
            self.portHandler,
            self.dxl_id,
            ADDR_OPERATING_MODE,
            PWM_CONTROL_MODE
        )

        self.packetHandler.write1ByteTxRx(
            self.portHandler,
            self.dxl_id,
            ADDR_TORQUE_ENABLE,
            TORQUE_ENABLE
        )

        self.write_pwm(0)
        time.sleep(0.1)

        mode, _, _ = self.packetHandler.read1ByteTxRx(
            self.portHandler,
            self.dxl_id,
            ADDR_OPERATING_MODE
        )

```

```

torque, _, _ = self.packetHandler.read1ByteTxRx(
    self.portHandler,
    self.dxl_id,
    ADDR_TORQUE_ENABLE
)

voltage_raw, _, _ = self.packetHandler.read2ByteTxRx(
    self.portHandler,
    self.dxl_id,
    ADDR_PRESENT_INPUT_VOLTAGE
)

print("PWM mode enabled")
print("mode:", mode)
print("torque:", torque)
print("input voltage:", voltage_raw / 10.0, "V")

def write_pwm(self, pwm):
    pwm = clip_pwm(pwm)
    pwm_unsigned = signed_pwm_to_unsigned(pwm)

    comm, err = self.packetHandler.write2ByteTxRx(
        self.portHandler,
        self.dxl_id,
        ADDR_GOAL_PWM,
        pwm_unsigned
    )

    return comm, err

def read_velocity_rad_s(self):
    vel_raw, _, _ = self.packetHandler.read4ByteTxRx(
        self.portHandler,
        self.dxl_id,
        ADDR_PRESENT_VELOCITY
    )

    return signed_velocity_to_rad_s(vel_raw)

def read_position_ticks(self):
    pos, _, _ = self.packetHandler.read4ByteTxRx(
        self.portHandler,
        self.dxl_id,
        ADDR_PRESENT_POSITION
    )

    return pos

def stop(self):
    try:
        self.write_pwm(0)
        time.sleep(0.1)

        self.packetHandler.write1ByteTxRx(
            self.portHandler,
            self.dxl_id,

```

```

        ADDR_TORQUE_ENABLE,
        TORQUE_DISABLE
    )

    except Exception:
        pass

    print("Dynamixel stopped")

def close(self):
    self.portHandler.closePort()
    print("Dynamixel port closed")

```

```

In [ ]: class VirtualEquilibriumFishingController:
    def __init__(
        self,
        sensor_reader,
        motor_interface,
        desired_tension_N,
        control_freq_Hz,
        max_duration_s
    ):
        self.sensor = sensor_reader
        self.motor = motor_interface

        self.desired_tension_N = float(desired_tension_N)
        self.dt_nominal = 1.0 / float(control_freq_Hz)

        self.max_duration_s = float(max_duration_s)
        self.loop_manager = FixedFrequencyLoopManager(control_freq_Hz)

        self.prev_time = None
        self.prev_tension_N = None
        self.prev_pwm_raw = 0

        self.virtual_target_pos = None

        self.control_enabled = False
        self.bite_timer_s = 0.0
        self.low_tension_timer_s = 0.0

        self.prev_region = "IDLE"

        self.time_history = []
        self.raw_history = []
        self.filtered_raw_history = []
        self.position_history = []
        self.virtual_target_history = []
        self.position_error_history = []
        self.tension_history = []
        self.raw_tension_history = []
        self.tension_error_history = []
        self.tension_rate_history = []
        self.controller_pwm_history = []
        self.release_ff_pwm_history = []
        self.pos_comp_pwm_history = []

```



```

self.pwm_history = []
self.region_history = []
self.supervisor_state_history = []
self.low_tension_timer_history = []

def apply_position_limits(self, pwm_raw, pos):
    limited = False
    reason = ""

    if pos <= MIN_POS_TICKS and pwm_raw < 0:
        pwm_raw = 0
        limited = True
        reason = "BACK_LIMIT"

    if pos >= MAX_POS_TICKS and pwm_raw > 0:
        pwm_raw = 0
        limited = True
        reason = "FORWARD_LIMIT"

    return pwm_raw, limited, reason

def slew_limit_pwm(self, pwm_raw):
    lower = self.prev_pwm_raw - MAX_PWM_STEP
    upper = self.prev_pwm_raw + MAX_PWM_STEP

    pwm_limited = int(round(max(lower, min(upper, pwm_raw))))

    return pwm_limited

def disable_control(self, current_pos):
    self.control_enabled = False
    self.bite_timer_s = 0.0
    self.low_tension_timer_s = 0.0
    self.virtual_target_pos = float(current_pos)
    self.prev_pwm_raw = 0
    self.prev_region = "IDLE"

def update_engagement_supervisor(self, tension_N, dt, pos):
    if not self.control_enabled:
        self.low_tension_timer_s = 0.0

        if tension_N > BITE_TRIGGER_TENSION_N:
            self.bite_timer_s += dt
        else:
            self.bite_timer_s = 0.0

        if self.bite_timer_s >= BITE_CONFIRM_TIME_S:
            self.control_enabled = True
            self.bite_timer_s = 0.0
            self.low_tension_timer_s = 0.0
            self.virtual_target_pos = float(pos)
            self.prev_pwm_raw = 0
            self.prev_region = "IDLE"
            return True

    self.virtual_target_pos = float(pos)

```

```

        self.prev_pwm_raw = 0
        return False

    if tension_N < LOW_TENSION_RECOVERY_N:
        self.low_tension_timer_s += dt
    elif tension_N > TENSION_RECOVERED_N:
        self.low_tension_timer_s = 0.0
    else:
        self.low_tension_timer_s = max(
            0.0,
            self.low_tension_timer_s - dt
        )

    low_tension_timeout = (
        tension_N < BITE_TRIGGER_TENSION_N
        and self.low_tension_timer_s >= LOW_TENSION_TIMEOUT_S
    )

    soft_back_limit_reached = (
        tension_N < LOW_TENSION_RECOVERY_N
        and pos <= SOFT_BACK_LIMIT_TICKS
    )

    if low_tension_timeout or soft_back_limit_reached:
        self.disable_control(pos)
        return False

    return True

def compute_raw_virtual_target(self, tension_error_N, current_pos):
    if tension_error_N > TENSION_DEADBAND_N:
        region = "HIGH"
        effective_error = tension_error_N - TENSION_DEADBAND_N
        offset = HIGH_TENSION_POS_GAIN_TICKS_PER_N * effective_error

    elif tension_error_N < -TENSION_DEADBAND_N:
        region = "LOW"
        effective_error = tension_error_N + TENSION_DEADBAND_N
        offset = LOW_TENSION_POS_GAIN_TICKS_PER_N * effective_error

    else:
        region = "DB"
        offset = 0.0

    offset = max(
        -MAX_TARGET_OFFSET_TICKS,
        min(MAX_TARGET_OFFSET_TICKS, offset)
    )

    if region == "DB":
        target_raw = float(current_pos)
    else:
        target_raw = CENTER_POS_TICKS + offset

    target_raw = max(
        MIN_POS_TICKS,

```

```

        min(MAX_POS_TICKS, target_raw)
    )

    return target_raw, region

def reset_virtual_target_on_region_reversal(self, region, current_pos):
    if not RESET_TARGET_ON_REGION_REVERSAL:
        return

    low_to_high = self.prev_region == "LOW" and region == "HIGH"
    high_to_low = self.prev_region == "HIGH" and region == "LOW"

    if low_to_high or high_to_low:
        self.virtual_target_pos = float(current_pos)
        self.prev_pwm_raw = 0

def update_virtual_target(self, target_raw, region, current_pos):
    if self.virtual_target_pos is None:
        self.virtual_target_pos = float(current_pos)

    if region == "DB":
        self.virtual_target_pos = float(current_pos)
        return self.virtual_target_pos

    target_smooth = (
        (1.0 - TARGET_ALPHA) * self.virtual_target_pos
        + TARGET_ALPHA * target_raw
    )

    lower = self.virtual_target_pos - MAX_TARGET_STEP_TICKS
    upper = self.virtual_target_pos + MAX_TARGET_STEP_TICKS

    target_limited = max(lower, min(upper, target_smooth))

    self.virtual_target_pos = float(target_limited)

    return self.virtual_target_pos

def position_error_to_pwm(self, position_error_ticks, motor_vel_rad_s, r
    if region == "DB":
        return 0

    if abs(position_error_ticks) < POSITION_ERROR_DEADBAND_TICKS:
        return 0

    pwm = (
        KP_POS_TO_PWM * position_error_ticks
        - KD_VEL_TO_PWM * motor_vel_rad_s
    )

    controller_pwm = int(round(pwm))
    controller_pwm = clip_pwm(controller_pwm)

    if region == "HIGH" and controller_pwm < 0:
        controller_pwm = 0

```

```

    if region == "LOW" and controller_pwm > 0:
        controller_pwm = 0

    if abs(position_error_ticks) > MIN_PWM_ENABLE_ERROR_TICKS:
        if abs(controller_pwm) > 0:
            controller_pwm = apply_min_effective_pwm(controller_pwm)

    return controller_pwm

def high_tension_release_feedforward_pwm(self, tension_error_N, region):
    if region != "HIGH":
        return 0

    over_tension = max(0.0, tension_error_N - TENSION_DEADBAND_N)

    release_pwm = int(round(
        HIGH_TENSION_RELEASE_PWM_GAIN * over_tension
    ))

    release_pwm = min(HIGH_TENSION_RELEASE_PWM_MAX, release_pwm)

    if release_pwm > 0:
        release_pwm = max(HIGH_TENSION_RELEASE_MIN_PWM, release_pwm)

    return release_pwm

def compute_final_pwm(self, controller_pwm, pos_comp, region):
    if region == "DB":
        pwm_raw = clip_pwm(pos_comp)
        return pwm_raw

    pwm_raw = controller_pwm + pos_comp
    pwm_raw = clip_pwm(pwm_raw)

    if region == "HIGH" and pwm_raw < 0:
        pwm_raw = 0
        if self.prev_pwm_raw < 0:
            self.prev_pwm_raw = 0

    if region == "LOW" and pwm_raw > 0:
        pwm_raw = 0
        if self.prev_pwm_raw > 0:
            self.prev_pwm_raw = 0

    if self.prev_pwm_raw * pwm_raw < 0:
        self.prev_pwm_raw = 0

    pwm_raw = self.slew_limit_pwm(pwm_raw)

    if region == "HIGH" and pwm_raw < 0:
        pwm_raw = 0

    if region == "LOW" and pwm_raw > 0:
        pwm_raw = 0

    return pwm_raw

```

```

def run_idle_output(self, pos):
    supervisor_state = "IDLE"
    region = "IDLE"

    target_pos = float(pos)
    position_error_ticks = 0.0
    controller_pwm = 0
    release_ff_pwm = 0
    pos_comp = 0
    pwm_raw = 0

    self.virtual_target_pos = float(pos)
    self.prev_pwm_raw = 0

    return (
        supervisor_state,
        region,
        target_pos,
        position_error_ticks,
        controller_pwm,
        release_ff_pwm,
        pos_comp,
        pwm_raw
    )

def run_control_output(self, tension_N, pos, motor_vel):
    if self.low_tension_timer_s > 0.0 and tension_N < LOW_TENSION_RECOVER:
        supervisor_state = "RECOVERY"
    else:
        supervisor_state = "CONTROL"

    tension_error_N = tension_N - self.desired_tension_N

    target_raw, region = self.compute_raw_virtual_target(
        tension_error_N=tension_error_N,
        current_pos=pos
    )

    self.reset_virtual_target_on_region_reversal(
        region=region,
        current_pos=pos
    )

    target_pos = self.update_virtual_target(
        target_raw=target_raw,
        region=region,
        current_pos=pos
    )

    position_error_ticks = target_pos - pos

    controller_pwm = self.position_error_to_pwm(
        position_error_ticks=position_error_ticks,
        motor_vel_rad_s=motor_vel,
        region=region
    )

```

```

    )

    release_ff_pwm = self.high_tension_release_feedforward_pwm(
        tension_error_N=tension_error_N,
        region=region
    )

    if region == "HIGH":
        controller_pwm = max(controller_pwm, release_ff_pwm)

    pos_comp = position_comp_pwm(pos)

    if region == "HIGH" and controller_pwm > 0 and pos_comp < 0:
        pos_comp = 0

    pwm_raw = self.compute_final_pwm(
        controller_pwm=controller_pwm,
        pos_comp=pos_comp,
        region=region
    )

    return (
        supervisor_state,
        region,
        target_pos,
        position_error_ticks,
        controller_pwm,
        release_ff_pwm,
        pos_comp,
        pwm_raw
    )

def run(self):
    print("Starting virtual-equilibrium fishing controller with engagement")
    print(f"Target tension: {self.desired_tension_N:.2f} N")
    print(f"Bite trigger: {BITE_TRIGGER_TENSION_N:.2f} N for {BITE_CONFIDENCE_N:.2f} s")
    print(f"Low-tension recovery threshold: {LOW_TENSION_RECOVERY_N:.2f} N")
    print(f"Recovered tension threshold: {TENSION_RECOVERED_N:.2f} N")
    print(f"Low-tension timeout: {LOW_TENSION_TIMEOUT_S:.2f} s")
    print(f"Soft back limit: {SOFT_BACK_LIMIT_TICKS} ticks")
    print(f"Deadband: +/- {TENSION_DEADBAND_N:.2f} N")
    print(f"Low-tension gain: {LOW_TENSION_POS_GAIN_TICKS_PER_N:.1f} ticks/N")
    print(f"High-tension gain: {HIGH_TENSION_POS_GAIN_TICKS_PER_N:.1f} ticks/N")
    print(f"High-tension release gain: {HIGH_TENSION_RELEASE_PWM_GAIN:.1f}")
    print(f"High-tension release max: {HIGH_TENSION_RELEASE_PWM_MAX}")
    print(f"High-tension release min: {HIGH_TENSION_RELEASE_MIN_PWM}")
    print(f"Max target offset: {MAX_TARGET_OFFSET_TICKS} ticks")
    print(f"Target alpha: {TARGET_ALPHA:.2f}")
    print(f"Max target step: {MAX_TARGET_STEP_TICKS} ticks")
    print(f"Reset target on reversal: {RESET_TARGET_ON_REGION_REVERSAL}")
    print(f"Safe position range: {MIN_POS_TICKS} to {MAX_POS_TICKS}")
    print(f"Position compensation: {USE_POSITION_COMP}, center={CENTER_F}")
    print("Press Ctrl+C to stop\n")

    t_start = time.time()
    self.prev_time = t_start

```

```

while True:
    now = time.time()
    t = now - t_start
    dt = max(now - self.prev_time, 1e-6)
    self.prev_time = now

    if t > self.max_duration_s:
        print("Max duration reached")
        self.motor.write_pwm(0)
        break

    raw, filtered_raw, tension_raw_N, line = self.sensor.read()

    if tension_raw_N is None:
        if line:
            print(f"Skipped sensor line: {line}")
            self.loop_manager.sleep()
            continue

    tension_N = max(0.0, tension_raw_N)

    if self.prev_tension_N is None:
        tension_rate = 0.0
    else:
        tension_rate = (tension_N - self.prev_tension_N) / dt

    self.prev_tension_N = tension_N

    pos = self.motor.read_position_ticks()
    motor_vel = self.motor.read_velocity_rad_s()

    engaged = self.update_engagement_supervisor(
        tension_N=tension_N,
        dt=dt,
        pos=pos
    )

    if engaged:
        (
            supervisor_state,
            region,
            target_pos,
            position_error_ticks,
            controller_pwm,
            release_ff_pwm,
            pos_comp,
            pwm_raw
        ) = self.run_control_output(
            tension_N=tension_N,
            pos=pos,
            motor_vel=motor_vel
        )
    else:
        (
            supervisor_state,

```

```

        region,
        target_pos,
        position_error_ticks,
        controller_pwm,
        release_ff_pwm,
        pos_comp,
        pwm_raw
    ) = self.run_idle_output(pos)

pwm_raw, limited, limit_reason = self.apply_position_limits(
    pwm_raw,
    pos
)

self.prev_pwm_raw = pwm_raw

comm, err = self.motor.write_pwm(pwm_raw)

tension_error_N = tension_N - self.desired_tension_N

self.time_history.append(t)
self.raw_history.append(raw)
self.filtered_raw_history.append(filtered_raw)
self.position_history.append(pos)
self.virtual_target_history.append(target_pos)
self.position_error_history.append(position_error_ticks)
self.tension_history.append(tension_N)
self.raw_tension_history.append(tension_raw_N)
self.tension_error_history.append(tension_error_N)
self.tension_rate_history.append(tension_rate)
self.controller_pwm_history.append(controller_pwm)
self.release_ff_pwm_history.append(release_ff_pwm)
self.pos_comp_pwm_history.append(pos_comp)
self.pwm_history.append(pwm_raw)
self.region_history.append(region)
self.supervisor_state_history.append(supervisor_state)
self.low_tension_timer_history.append(self.low_tension_timer_s)

self.prev_region = region

limit_msg = f" | {limit_reason}" if limited else ""

print(
    f"t={t:.2f}s | "
    f"sup={supervisor_state} | "
    f"region={region} | "
    f"pos={pos} | "
    f"target={target_pos:.1f} | "
    f"posErr={position_error_ticks:.1f} | "
    f"raw={raw} | "
    f"filtered={filtered_raw:.1f} | "
    f"zero={self.sensor.zero_offset:.1f} | "
    f"tension={tension_N:.3f} N | "
    f"rawT={tension_raw_N:.3f} N | "
    f"terr={tension_error_N:.3f} N | "
    f"dT={tension_rate:.2f} N/s | "

```



```

        f"lowTimer={self.low_tension_timer_s:.2f}s | "
        f"v_motor={motor_vel:.3f} | "
        f"releaseFF={release_ff_pwm} | "
        f"ctrlPWM={controller_pwm} | "
        f"posComp={pos_comp} | "
        f"PWM={pwm_raw} | "
        f"dxl_err={err}"
        f"{limit_msg}"
    )

    self.loop_manager.sleep()

```

```

In [ ]: sensor = LoadCellSerialReader(
    port=ARDUINO_PORT,
    baud=ARDUINO_BAUD,
    timeout_s=ARDUINO_TIMEOUT_S,
    filter_window=FILTER_WINDOW,
    tare_samples=TARE_SAMPLES,
    sensitivity_N_per_count=SENSITIVITY_N_PER_COUNT
)

motor = DynamixelPWMInterface(
    dxl_id=DXL_ID,
    port=DXL_PORT,
    baud=DXL_BAUD
)

controller = None

try:
    sensor.open()

    motor.open()
    motor.set_pwm_limit(885)
    motor.enable_pwm_mode()

    controller = VirtualEquilibriumFishingController(
        sensor_reader=sensor,
        motor_interface=motor,
        desired_tension_N=TARGET_TENSION_N,
        control_freq_Hz=CONTROL_FREQ_HZ,
        max_duration_s=MAX_DURATION_S
    )

    controller.run()

except KeyboardInterrupt:
    print("\nStopped by user")

finally:
    try:
        motor.stop()
        motor.close()
    except Exception:
        pass

```

```

try:
    sensor.close()
except Exception:
    pass

print("Trial finished")

```

```

In [ ]: if controller is None or len(controller.time_history) == 0:
        print("No data to plot")
    else:
        t = np.asarray(controller.time_history)
        tension = np.asarray(controller.tension_history)
        raw_tension = np.asarray(controller.raw_tension_history)
        tension_error = np.asarray(controller.tension_error_history)
        tension_rate = np.asarray(controller.tension_rate_history)

        pos = np.asarray(controller.position_history)
        target = np.asarray(controller.virtual_target_history)
        pos_error = np.asarray(controller.position_error_history)

        pwm = np.asarray(controller.pwm_history)
        ctrl_pwm = np.asarray(controller.controller_pwm_history)
        release_ff = np.asarray(controller.release_ff_pwm_history)
        pos_comp = np.asarray(controller.pos_comp_pwm_history)
        low_timer = np.asarray(controller.low_tension_timer_history)

        plt.figure(figsize=(8, 4))
        plt.plot(t, tension, label="Tension")
        plt.plot(t, raw_tension, label="Raw tension", alpha=0.5)
        plt.axhline(TARGET_TENSION_N, linestyle="--", label="Target")
        plt.axhline(BITE_TRIGGER_TENSION_N, linestyle="-. ", label="Bite trigger")
        plt.axhline(LOW_TENSION_RECOVERY_N, linestyle=":", label="Recovery thresh")
        plt.axhline(TENSION_RECOVERED_N, linestyle=":", label="Recovered thresh")
        plt.xlabel("Time [s]")
        plt.ylabel("Tension [N]")
        plt.title("Tension vs Time")
        plt.legend()
        plt.grid(True)
        plt.show()

        plt.figure(figsize=(8, 4))
        plt.plot(t, tension_error, label="Tension error")
        plt.axhline(0.0, linestyle="--", label="Zero error")
        plt.axhline(TENSION_DEADBAND_N, linestyle=":", label="Upper deadband")
        plt.axhline(-TENSION_DEADBAND_N, linestyle=":", label="Lower deadband")
        plt.xlabel("Time [s]")
        plt.ylabel("Tension error [N]")
        plt.title("Tension Error")
        plt.legend()
        plt.grid(True)
        plt.show()

        plt.figure(figsize=(8, 4))
        plt.plot(t, pos, label="Motor position")
        plt.plot(t, target, label="Virtual target position")
        plt.axhline(MIN_POS_TICKS, linestyle="--", label="Back hard limit")

```

```

plt.axhline(SOFT_BACK_LIMIT_TICKS, linestyle="-. ", label="Back soft limit")
plt.axhline(MAX_POS_TICKS, linestyle="--", label="Forward limit")
plt.axhline(CENTER_POS_TICKS, linestyle=":", label="Center")
plt.xlabel("Time [s]")
plt.ylabel("Position [ticks]")
plt.title("Position and Virtual Target")
plt.legend()
plt.grid(True)
plt.show()

```

```

plt.figure(figsize=(8, 4))
plt.plot(t, pos_error, label="Position error")
plt.axhline(0.0, linestyle="--", label="Zero error")
plt.axhline(POSITION_ERROR_DEADBAND_TICKS, linestyle=":", label="Positive error limit")
plt.axhline(-POSITION_ERROR_DEADBAND_TICKS, linestyle=":", label="Negative error limit")
plt.xlabel("Time [s]")
plt.ylabel("Position error [ticks]")
plt.title("Virtual Target Tracking Error")
plt.legend()
plt.grid(True)
plt.show()

```

```

plt.figure(figsize=(8, 4))
plt.plot(t, pwm, label="Final PWM")
plt.plot(t, ctrl_pwm, label="Controller PWM")
plt.plot(t, release_ff, label="High-tension release FF")
plt.plot(t, pos_comp, label="Position compensation PWM")
plt.xlabel("Time [s]")
plt.ylabel("PWM")
plt.title("PWM Components")
plt.legend()
plt.grid(True)
plt.show()

```

```

plt.figure(figsize=(8, 4))
plt.plot(t, low_timer, label="Low-tension timer")
plt.axhline(LOW_TENSION_TIMEOUT_S, linestyle="--", label="Timeout")
plt.xlabel("Time [s]")
plt.ylabel("Timer [s]")
plt.title("Low-Tension Recovery Timer")
plt.legend()
plt.grid(True)
plt.show()

```

```

plt.figure(figsize=(8, 4))
plt.plot(t, tension_rate, label="Tension rate")
plt.axhline(0.0, linestyle="--", label="Zero rate")
plt.xlabel("Time [s]")
plt.ylabel("dT/dt [N/s]")
plt.title("Tension Rate")
plt.legend()
plt.grid(True)
plt.show()

```

```

print("Supervisor states used:", sorted(set(controller.supervisor_state)))
print("Regions used:", sorted(set(controller.region_history)))

```

```

In [ ]: # =====
# Supplementary final report plots
# Cleaner selected figures for Results section
# =====

import numpy as np
import matplotlib.pyplot as plt

if controller is None or len(controller.time_history) == 0:
    print("No data to plot")
else:
    # =====
    # Time windows
    # =====
    t_min = 0.0
    t_max_main = 20.0      # tension error, PWM, region
    t_max_position = 14.0  # position response

    t = np.asarray(controller.time_history)

    mask_main = (t >= t_min) & (t <= t_max_main)
    mask_pos = (t >= t_min) & (t <= t_max_position)

    t_main = t[mask_main]
    t_pos = t[mask_pos]

    # =====
    # Unit conversion: ticks to degrees
    # CENTER_POS_TICKS is treated as 0 deg reference
    # =====
    TICKS_PER_REV = 4096.0

    def ticks_to_deg(ticks):
        return (np.asarray(ticks) - CENTER_POS_TICKS) * 360.0 / TICKS_PER_REV

    def tick_error_to_deg(error_ticks):
        return np.asarray(error_ticks) * 360.0 / TICKS_PER_REV

    # =====
    # 1. Tension Error
    # Better performance/error assessment than position error
    # =====
    tension_error = np.asarray(controller.tension_error_history)[mask_main]

    plt.figure(figsize=(10, 5))
    plt.plot(t_main, tension_error, label="Tension error", linewidth=2)
    plt.axhline(0.0, linestyle="--", label="Zero error")
    plt.axhline(TENSION_DEADBAND_N, linestyle=":", label="Upper deadband")
    plt.axhline(-TENSION_DEADBAND_N, linestyle=":", label="Lower deadband")

    plt.xlabel("Time [s]")
    plt.ylabel("Tension error [N]")
    plt.title("Tension Regulation Error")
    plt.xlim(t_min, t_max_main)
    plt.grid(True)

```

```

plt.legend(loc="best")
plt.tight_layout()
plt.savefig("report_tension_error.png", dpi=300)
plt.show()

# =====
# 2. Rod Joint Position Response in Degrees
# Use degree instead of ticks to satisfy report requirement
# =====
pos_ticks = np.asarray(controller.position_history)[mask_pos]
target_ticks = np.asarray(controller.virtual_target_history)[mask_pos]

pos_deg = ticks_to_deg(pos_ticks)
target_deg = ticks_to_deg(target_ticks)

min_pos_deg = ticks_to_deg(MIN_POS_TICKS)
soft_back_deg = ticks_to_deg(SOFT_BACK_LIMIT_TICKS)
max_pos_deg = ticks_to_deg(MAX_POS_TICKS)
center_deg = 0.0

plt.figure(figsize=(10, 5))
plt.plot(t_pos, pos_deg, label="Rod joint position", linewidth=2)
plt.plot(t_pos, target_deg, label="Virtual target", linewidth=2)

plt.axhline(min_pos_deg, linestyle="--", label="Back hard limit")
plt.axhline(soft_back_deg, linestyle="-. ", label="Back soft limit")
plt.axhline(max_pos_deg, linestyle="--", label="Forward limit")
plt.axhline(center_deg, linestyle=":", label="Center")

plt.xlabel("Time [s]")
plt.ylabel("Rod joint position [deg]")
plt.title("Rod Joint Position Response to Virtual Target")
plt.xlim(t_min, t_max_position)
plt.grid(True)
plt.legend(loc="lower right")
plt.tight_layout()
plt.savefig("report_position_response_deg.png", dpi=300)
plt.show()

# =====
# 3. Simplified Control Effort / PWM
# Cleaner than plotting every component
# =====
pwm = np.asarray(controller.pwm_history)[mask_main]
release_ff = np.asarray(controller.release_ff_pwm_history)[mask_main]
pos_comp = np.asarray(controller.pos_comp_pwm_history)[mask_main]

plt.figure(figsize=(10, 5))
plt.plot(t_main, pwm, label="Final PWM command", linewidth=2)
plt.plot(t_main, release_ff, label="High-tension release feedforward", 1
plt.plot(t_main, pos_comp, label="Position compensation", linewidth=1.5)
plt.axhline(0.0, linestyle="--", label="Zero PWM")

plt.xlabel("Time [s]")
plt.ylabel("PWM")
plt.title("Control Effort")

```

```

plt.xlim(t_min, t_max_main)
plt.grid(True)
plt.legend(loc="best")
plt.tight_layout()
plt.savefig("report_control_effort_pwm.png", dpi=300)
plt.show()

# =====
# 4. Controller Region Over Time
# Shows IDLE / LOW / DB / HIGH switching logic
# =====
region_map = {
    "IDLE": 0,
    "LOW": 1,
    "DB": 2,
    "HIGH": 3,
    "RECOVERY": 4,
}

region_history = np.asarray(controller.region_history)[mask_main]
region_num = np.asarray([region_map.get(str(r), -1) for r in region_hist

plt.figure(figsize=(10, 4))
plt.step(t_main, region_num, where="post", label="Controller region", li

plt.yticks(
    [0, 1, 2, 3, 4],
    ["IDLE", "LOW", "DB", "HIGH", "RECOVERY"]
)

plt.xlabel("Time [s]")
plt.ylabel("Controller region")
plt.title("Controller Region Over Time")
plt.xlim(t_min, t_max_main)
plt.grid(True)
plt.legend(loc="best")
plt.tight_layout()
plt.savefig("report_controller_region.png", dpi=300)
plt.show()

print("Saved report-ready supplementary figures:")
print("report_tension_error.png")
print("report_position_response_deg.png")
print("report_control_effort_pwm.png")
print("report_controller_region.png")

```

```

In [ ]: # =====
# Supervisor State Over Time
# Shows IDLE / CONTROL / RECOVERY state-machine behavior
# =====

import numpy as np
import matplotlib.pyplot as plt

if controller is None or len(controller.time_history) == 0:
    print("No data to plot")

```

```

else:
    t_min = 0.0
    t_max = 20.0

    t = np.asarray(controller.time_history)
    mask = (t >= t_min) & (t <= t_max)

    t_plot = t[mask]
    supervisor_history = np.asarray(controller.supervisor_state_history)[mas

    supervisor_map = {
        "IDLE": 0,
        "CONTROL": 1,
        "RECOVERY": 2,
    }

    supervisor_num = np.asarray([
        supervisor_map.get(str(s), -1) for s in supervisor_history
    ])

    plt.figure(figsize=(10, 4))
    plt.step(
        t_plot,
        supervisor_num,
        where="post",
        label="Supervisor state",
        linewidth=2
    )

    plt.yticks(
        [0, 1, 2],
        ["IDLE", "CONTROL", "RECOVERY"]
    )

    plt.xlabel("Time [s]")
    plt.ylabel("Supervisor state")
    plt.title("Supervisor State Over Time")
    plt.xlim(t_min, t_max)
    plt.grid(True)
    plt.legend(loc="best")
    plt.tight_layout()
    plt.savefig("report_supervisor_state.png", dpi=300)
    plt.show()

    print("Saved: report_supervisor_state.png")

```

```

In [ ]: # =====
# Paper-ready plotting cell
# Compatible with current notebook structure
# - No raw tension
# - Actual joint angle only
# - Position error also plotted in degrees
# - Saves figures + CSV using numpy only
# =====

import numpy as np

```

```

import matplotlib.pyplot as plt
import csv

if controller is None or len(controller.time_history) == 0:
    print("No data to plot. Please run the controller first.")
else:
    # =====
    # Time window
    # =====
    t_min = 0.0
    t_max = 20.0

    t_all = np.asarray(controller.time_history)
    mask = (t_all >= t_min) & (t_all <= t_max)

    t = t_all[mask]

    # =====
    # Load histories
    # =====
    tension = np.asarray(controller.tension_history)[mask]
    tension_error = np.asarray(controller.tension_error_history)[mask]

    pos_ticks = np.asarray(controller.position_history)[mask]
    target_ticks = np.asarray(controller.virtual_target_history)[mask]
    pos_error_ticks = np.asarray(controller.position_error_history)[mask]

    pwm = np.asarray(controller.pwm_history)[mask]
    ctrl_pwm = np.asarray(controller.controller_pwm_history)[mask]
    release_ff = np.asarray(controller.release_ff_pwm_history)[mask]
    pos_comp = np.asarray(controller.pos_comp_pwm_history)[mask]

    region_history = np.asarray(controller.region_history)[mask]

    if hasattr(controller, "supervisor_state_history"):
        supervisor_history = np.asarray(controller.supervisor_state_history)
    else:
        supervisor_history = None

    if hasattr(controller, "low_tension_timer_history"):
        low_timer = np.asarray(controller.low_tension_timer_history)[mask]
    else:
        low_timer = None

    # =====
    # Convert ticks to degrees
    # 4096 ticks = 360 deg
    # CENTER_POS_TICKS is treated as 0 deg
    # =====
    TICKS_PER_REV = 4096.0

    def ticks_to_deg(ticks):
        return (np.asarray(ticks) - CENTER_POS_TICKS) * 360.0 / TICKS_PER_REV

    def tick_error_to_deg(error_ticks):
        return np.asarray(error_ticks) * 360.0 / TICKS_PER_REV

```



```

joint_angle_deg = ticks_to_deg(pos_ticks)
target_angle_deg = ticks_to_deg(target_ticks)
pos_error_deg = tick_error_to_deg(pos_error_ticks)

min_angle_deg = ticks_to_deg(MIN_POS_TICKS)
soft_back_angle_deg = ticks_to_deg(SOFT_BACK_LIMIT_TICKS)
max_angle_deg = ticks_to_deg(MAX_POS_TICKS)
center_angle_deg = 0.0

pos_error_deadband_deg = POSITION_ERROR_DEADBAND_TICKS * 360.0 / TICKS_F

# =====
# Save CSV without pandas
# =====
csv_filename = "paper_trial_log.csv"

with open(csv_filename, "w", newline="") as f:
    writer = csv.writer(f)

    header = [
        "time_s",
        "tension_N",
        "tension_error_N",
        "position_ticks",
        "target_ticks",
        "joint_angle_deg",
        "target_angle_deg",
        "position_error_ticks",
        "position_error_deg",
        "final_pwm",
        "controller_pwm",
        "release_ff_pwm",
        "position_comp_pwm",
        "region",
    ]

    if supervisor_history is not None:
        header.append("supervisor_state")

    if low_timer is not None:
        header.append("low_tension_timer_s")

    writer.writerow(header)

    for i in range(len(t)):
        row = [
            t[i],
            tension[i],
            tension_error[i],
            pos_ticks[i],
            target_ticks[i],
            joint_angle_deg[i],
            target_angle_deg[i],
            pos_error_ticks[i],
            pos_error_deg[i],

```

```

        pwm[i],
        ctrl_pwm[i],
        release_ff[i],
        pos_comp[i],
        region_history[i],
    ]

    if supervisor_history is not None:
        row.append(supervisor_history[i])

    if low_timer is not None:
        row.append(low_timer[i])

    writer.writerow(row)

# =====
# 1. Tension response
# =====
plt.figure(figsize=(10, 5))
plt.plot(t, tension, label="Measured tension", linewidth=2)

plt.axhline(TARGET_TENSION_N, linestyle="--", label="Target tension")
plt.axhline(BITE_TRIGGER_TENSION_N, linestyle="-. ", label="Bite trigger")
plt.axhline(LOW_TENSION_RECOVERY_N, linestyle=":", label="Low-tension th")
plt.axhline(TENSION_RECOVERED_N, linestyle=":", label="Recovered threshc")

plt.xlabel("Time [s]")
plt.ylabel("Tension [N]")
plt.title("Tension Response During Hardware Trial")
plt.xlim(t_min, t_max)
plt.ylim(0, max(6.0, float(np.max(tension)) + 0.5))
plt.grid(True)
plt.legend(loc="best")
plt.tight_layout()
plt.savefig("paper_01_tension_response.png", dpi=300)
plt.show()

# =====
# 2. Tension error
# =====
plt.figure(figsize=(10, 5))
plt.plot(t, tension_error, label="Tension error", linewidth=2)

plt.axhline(0.0, linestyle="--", label="Zero error")
plt.axhline(TENSION_DEADBAND_N, linestyle=":", label="Upper deadband")
plt.axhline(-TENSION_DEADBAND_N, linestyle=":", label="Lower deadband")

plt.xlabel("Time [s]")
plt.ylabel("Tension error [N]")
plt.title("Tension Regulation Error")
plt.xlim(t_min, t_max)
plt.grid(True)
plt.legend(loc="best")
plt.tight_layout()
plt.savefig("paper_02_tension_error.png", dpi=300)
plt.show()

```

```

# =====
# 3. Actual rod joint angle only
# Do not plot virtual target in the main joint-angle figure
# =====
plt.figure(figsize=(10, 5))
plt.plot(t, joint_angle_deg, label="Rod joint angle", linewidth=2)

plt.axhline(min_angle_deg, linestyle="--", label="Back hard limit")
plt.axhline(soft_back_angle_deg, linestyle="-. ", label="Back soft limit")
plt.axhline(max_angle_deg, linestyle="--", label="Forward limit")
plt.axhline(center_angle_deg, linestyle=":", label="Center")

plt.xlabel("Time [s]")
plt.ylabel("Rod joint angle [deg]")
plt.title("Rod Joint Angle During Tension Regulation")
plt.xlim(t_min, t_max)
plt.grid(True)
plt.legend(loc="best")
plt.tight_layout()
plt.savefig("paper_03_actual_joint_angle_deg.png", dpi=300)
plt.show()

# =====
# 4. Position error in degrees
# Backup / optional figure
# =====
plt.figure(figsize=(10, 5))
plt.plot(t, pos_error_deg, label="Position error", linewidth=2)

plt.axhline(0.0, linestyle="--", label="Zero error")
plt.axhline(pos_error_deadband_deg, linestyle=":", label="Positive deadband")
plt.axhline(-pos_error_deadband_deg, linestyle=":", label="Negative deadband")

plt.xlabel("Time [s]")
plt.ylabel("Position error [deg]")
plt.title("Virtual Target Position Error")
plt.xlim(t_min, t_max)
plt.grid(True)
plt.legend(loc="best")
plt.tight_layout()
plt.savefig("paper_04_position_error_deg_backup.png", dpi=300)
plt.show()

# =====
# 5. Control effort / PWM
# =====
plt.figure(figsize=(10, 5))
plt.plot(t, pwm, label="Final PWM command", linewidth=2)
plt.plot(t, release_ff, label="High-tension release feedforward", linewidth=2)
plt.plot(t, pos_comp, label="Position compensation", linewidth=1.5)
plt.axhline(0.0, linestyle="--", label="Zero PWM")

plt.xlabel("Time [s]")
plt.ylabel("PWM")
plt.title("Control Effort During Tension Regulation")

```

```

plt.xlim(t_min, t_max)
plt.grid(True)
plt.legend(loc="best")
plt.tight_layout()
plt.savefig("paper_05_control_effort_pwm.png", dpi=300)
plt.show()

# =====
# 6. Controller region
# =====

region_map = {
    "IDLE": 0,
    "LOW": 1,
    "DB": 2,
    "HIGH": 3,
    "RECOVERY": 4,
}

region_num = np.asarray([
    region_map.get(str(r), -1) for r in region_history
])

plt.figure(figsize=(10, 4))
plt.step(t, region_num, where="post", label="Controller region", linewidth=2)

plt.yticks(
    [0, 1, 2, 3, 4],
    ["IDLE", "LOW", "DB", "HIGH", "RECOVERY"]
)

plt.xlabel("Time [s]")
plt.ylabel("Controller region")
plt.title("Controller Region Over Time")
plt.xlim(t_min, t_max)
plt.grid(True)
plt.legend(loc="best")
plt.tight_layout()
plt.savefig("paper_06_controller_region.png", dpi=300)
plt.show()

# =====
# 7. Supervisor state
# =====

if supervisor_history is not None:
    supervisor_map = {
        "IDLE": 0,
        "CONTROL": 1,
        "RECOVERY": 2,
    }

    supervisor_num = np.asarray([
        supervisor_map.get(str(s), -1) for s in supervisor_history
    ])

    plt.figure(figsize=(10, 4))
    plt.step(t, supervisor_num, where="post", label="Supervisor state",

```

```

plt.yticks(
    [0, 1, 2],
    ["IDLE", "CONTROL", "RECOVERY"]
)

plt.xlabel("Time [s]")
plt.ylabel("Supervisor state")
plt.title("Supervisor State Over Time")
plt.xlim(t_min, t_max)
plt.grid(True)
plt.legend(loc="best")
plt.tight_layout()
plt.savefig("paper_07_supervisor_state.png", dpi=300)
plt.show()

# =====
# 8. Low-tension recovery timer
# Only useful when recovery behavior occurs
# =====
if low_timer is not None:
    plt.figure(figsize=(10, 5))
    plt.plot(t, low_timer, label="Low-tension timer", linewidth=2)
    plt.axhline(LOW_TENSION_TIMEOUT_S, linestyle="--", label="Timeout th

    plt.xlabel("Time [s]")
    plt.ylabel("Timer [s]")
    plt.title("Low-Tension Recovery Timer")
    plt.xlim(t_min, t_max)
    plt.grid(True)
    plt.legend(loc="best")
    plt.tight_layout()
    plt.savefig("paper_08_low_tension_timer.png", dpi=300)
    plt.show()

print("Saved paper-ready figures:")
print("paper_01_tension_response.png")
print("paper_02_tension_error.png")
print("paper_03_actual_joint_angle_deg.png")
print("paper_04_position_error_deg_backup.png")
print("paper_05_control_effort_pwm.png")
print("paper_06_controller_region.png")
print("paper_07_supervisor_state.png")
print("paper_08_low_tension_timer.png")
print("Saved CSV:")
print(csv_filename)

```